Project Proposal

Reproduction and Extension of SafeSplit

Antonio Aparicio Andrew Mullen Mateo Lopez

1. Paper and Project Goal

We propose to reproduce and extend *SafeSplit: A Novel Defense Against Client Side Backdoor Attacks in Split Learning*. The paper studies U shaped Split Learning, where the client keeps the head and tail of the model, while the server keeps the backbone. SafeSplit is a server side defense against client side backdoor attacks. It detects suspicious backbone updates using two complementary signals: a static frequency based analysis and a dynamic rotational distance analysis. If the current checkpoint appears poisoned, the server rolls back to the latest benign checkpoint before allowing the next client to continue training.

Our goal is not to reproduce the entire paper, but a focused subset of its main results in a simplified implementation. In particular, we target the core claim of the paper: **SafeSplit can strongly reduce Backdoor Accuracy while preserving Main Task Accuracy**. We then extend the method with a lightweight **client trust score** stored on the server.

Main idea:

- reproduce the main SafeSplit behavior in a clean PyTorch implementation,
- focus on a small set of tables and figures from the paper,
- add a simple and justified extension based on client reliability history.

2. Focused Reproduction Targets

We focus on a subset of targets that captures the main contribution.

2.1 Target 1: Main CIFAR 10 Results from Table II

Our first target is the CIFAR 10 part of Table II, since it directly tests the central claim of the paper. We focus on both the semantic trigger and the pixel trigger settings.

Table 1: Primary reproduction target from the paper: CIFAR 10 results from Table II.

Attack Setting	No Defense BA	No Defense MA	SafeSplit BA	SafeSplit MA
Semantic Trigger	59.3	66.6	0.0	62.7
Pixel Trigger	100.0	66.6	0.3	66.4

We aim to recover the same qualitative behavior:

- high BA without defense,
- very low BA with SafeSplit,
- only a limited drop in MA.

2.2 Target 2: Different Data Distributions from Table III

Our second target is Table III, which studies different IID rates on CIFAR 10.

Table 2: Secondary reproduction target from the paper: Table III on CIFAR 10 with different IID rates.

IID Rate	No Defense BA	No Defense MA	SafeSplit BA	SafeSplit MA
0.6	44.7	59.5	0.0	57.5
0.8	59.3	66.6	0.0	62.7
1.0	60.7	69.4	0.0	68.1

This lets us test whether SafeSplit remains effective under different client data distributions.

2.3 Target 3: Reduced Baseline Comparison Inspired by Figure 7

The paper also compares SafeSplit against adapted baselines such as KRUM, Differential Privacy, and FreqFed. We propose a reduced comparison inspired by Figure 7.

Our reduced comparison will include:

- No Defense,
- SafeSplit,
- at least one or two simplified baselines, with priority given to Differential Privacy and a KRUM style distance baseline.

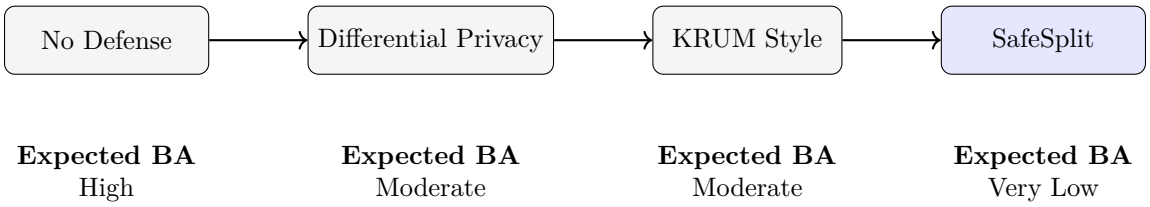

Figure 1: Reduced baseline comparison inspired by Figure 7.

3. System Setting and Implementation

We will implement a U shaped Split Learning system in PyTorch. Each client will hold:

- a head subnetwork,
- a tail subnetwork,
- its own private local data.

The server will hold:

- the backbone subnetwork,
- the SafeSplit analysis logic,
- the checkpoint history,
- the rollback mechanism,
- the trust score extension.

The implementation will use:

- PyTorch and torchvision,
- CIFAR 10 as the main dataset,
- optionally MNIST for debugging,
- a simple CNN or a small ResNet style split model,
- sequential training across clients.

We will build our own simplified implementation of SafeSplit so that the pipeline, attack setting, evaluation, and extension are fully controlled and easy to modify.

4. Motivation for the Extension

Our extension is a client trust score stored on the server.

The original SafeSplit method keeps a history of model checkpoints and uses rollback to recover a benign checkpoint. However, it does not explicitly maintain a persistent reliability estimate for each client across rounds.

Identified Limitation

SafeSplit remembers model history, but it does not explicitly remember client reliability history.

This matters because:

- a client may behave suspiciously multiple times across training,
- a client may alternate between benign and malicious behavior,
- repeated mild anomalies may still be informative.

For each client c, the server stores a trust score $T_c \in [0, 1]$.

- all clients start with high trust,
- accepted updates keep the score high or increase it slightly,
- suspicious updates decrease the score.

The trust score is then used to make the defense stricter for repeatedly suspicious clients.

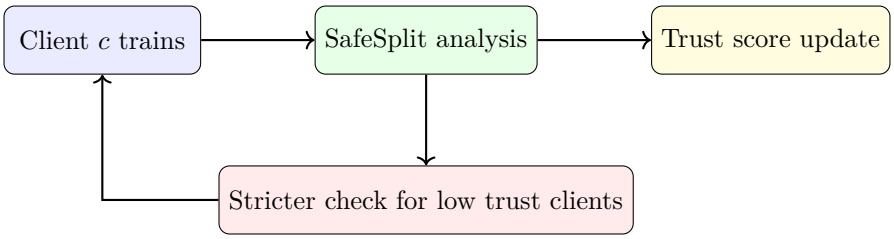

Figure 2: Conceptual view of the proposed trust score extension.

5. Methodology

We first implement a clean U shaped split learning pipeline without any attack, to verify that sequential client training works correctly and that the split model trains stably.

We then simulate client side backdoor attacks by making one or more clients malicious. Malicious clients poison a fraction of their local data using a trigger patch and a target label.

We evaluate:

- Main Task Accuracy (MA) on clean test data,
- Backdoor Accuracy (BA) on triggered test data.

We reproduce the main SafeSplit components:

- static frequency based analysis,
- dynamic rotational distance metric,
- checkpoint history,
- rollback to the latest benign checkpoint.

After reproducing the base method, we compare:

- No Defense,
- SafeSplit,
- SafeSplit + Trust Score.

6. Execution Plan

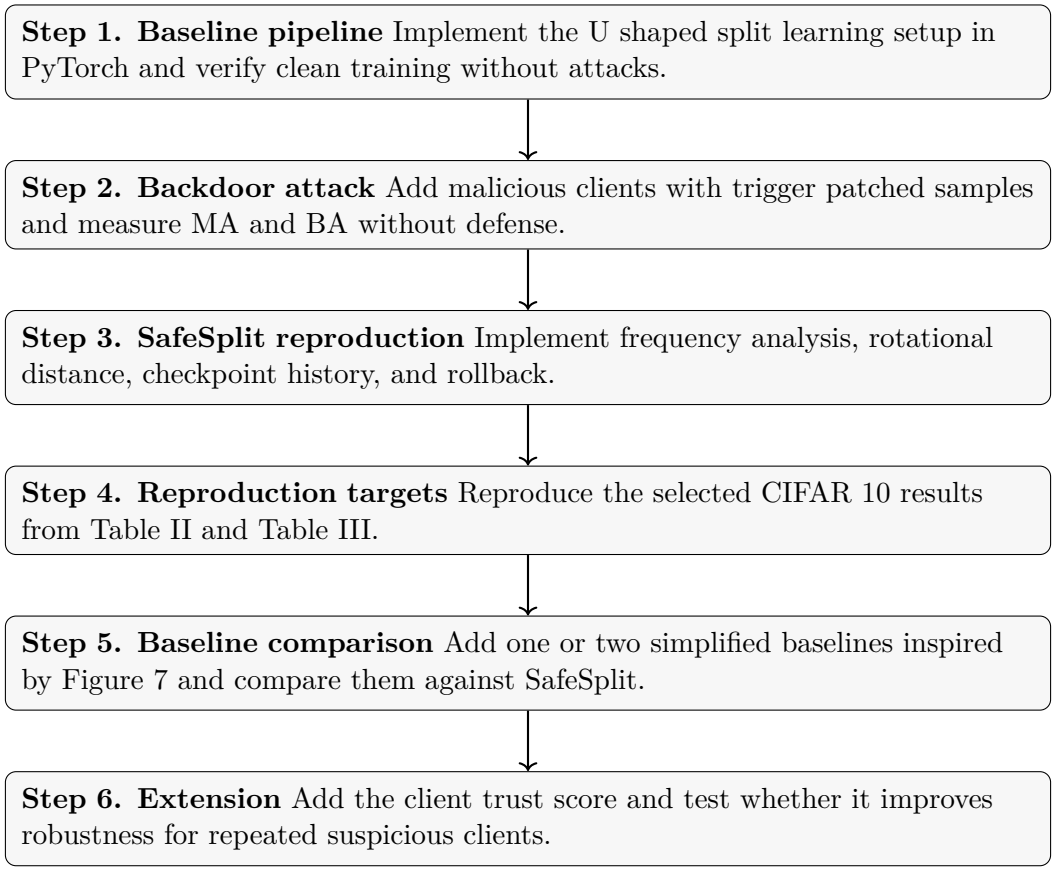

Figure 3: Execution plan for the project.

7. Overall Workflow

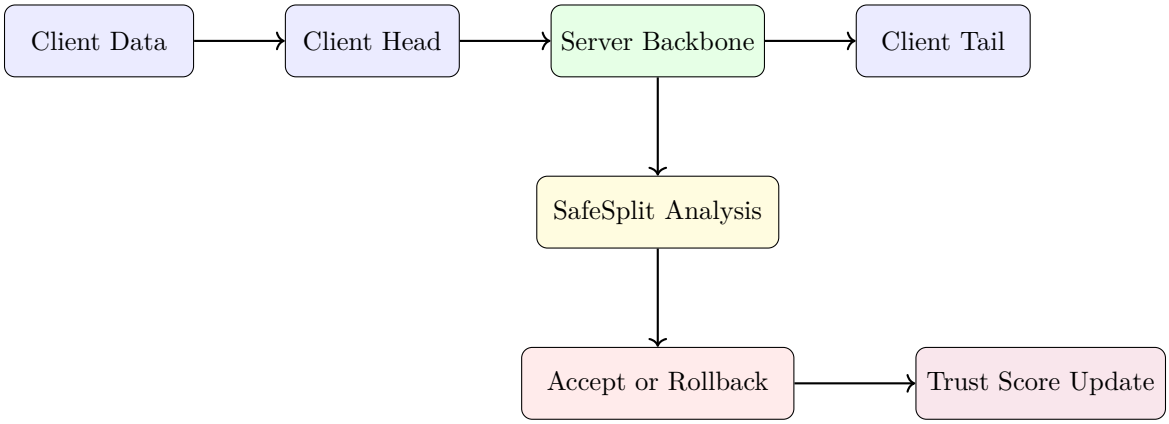

Figure 4: Overall workflow of the reproduced system and the proposed extension.

8. Expected Outcome

We expect the project to show:

- high Backdoor Accuracy without defense,

- strong BA reduction with SafeSplit,
- reasonable preservation of Main Task Accuracy,
- additional robustness from the trust score when suspicious behavior repeats.

Our goal is to reproduce the behavioral trend of the method rather than exact numerical matching, since our setup will be simplified and our implementation is independent.

9. Conclusion

This project proposes a focused reproduction of SafeSplit in a simplified U shaped Split Learning setting. The reproduction is centered on selected tables and a reduced baseline comparison that capture the paper's main contribution. The extension is motivated by a clear limitation of the original method: SafeSplit keeps checkpoint history, but not explicit client reliability history. By adding a lightweight trust score on the server, we aim to test whether persistent client level memory can further improve robustness.